

UNIVERSIDAD CARLOS III



Sistemas Distribuidos

Ejercicio Evaluable 1: Colas de Mensajes

CURSO 2025 - 2026

WENJIE CHEN → 100522255@alumnos.uc3m.es
Grupo 85, Ingeniería Informática

XIYA YE → 100522159@alumnos.uc3m.es
Grupo 85, Ingeniería Informática

Índice

Índice.....	2
1. Diseño.....	3
1.1. Versión A: No distribuida.....	3
1.2. Versión B: Distribuida utilizando colas de mensajes POSIX.....	3
2. Compilación y Ejecución.....	4
2.1. Makefile.....	4
2.2. Ejecución en la versión no distribuida.....	4
2.3. Ejecución en la versión distribuida con colas de mensajes POSIX.....	4
3. Batería de Pruebas.....	5
3.1. Pruebas de Funcionalidad (Pruebas 1-6).....	5
3.2. Pruebas de Control de Errores y Límites (Pruebas 7-15).....	5

1. Diseño

Para el diseño de la práctica, se ha dividido en dos fases diferentes, empezando por una versión básica a una versión mejorada mediante colas de mensajes POSIX.

1.1. Versión A: No distribuida

En esta versión, la aplicación cliente (*app-cliente.c*) interactúa directamente con la biblioteca dinámica (*libclaves.so*), que la hemos generado a partir de *claves.c*.

Para el almacenamiento de las tuplas, hemos decidido utilizar un sistema que usa ficheros. Para ello, se crea un directorio raíz *./ALMACEN*. Así, cada vez que se inserta una nueva tupla, el sistema genera un nuevo archivo de texto dentro de esta carpeta, con el nombre de la clave *key*. La lógica principal consiste en asignar a cada clave (*key*) un archivo individual dentro de dicha carpeta, utilizando funciones del sistema como *sprintf* para construir las rutas y *access* para verificar si un dato ya existe o no antes de hacer la operación.

Dentro de cada archivo, hemos guardado los valores de la tupla en el orden de llegada, separados por un fin de línea (*'\n'*). Primero la cadena *value1*, seguida de la dimensión del vector (*N_value2*), los valores decimales del mismo y *value3*, que viene dado por una estructura de datos. Al hacerlo de esta manera, nos permite que a la hora hacer una lectura (*get_value*) o una modificación (*modify_value*) nos sea más sencillo, usando funciones como *fopen* o *fscanf*.

1.2. Versión B: Distribuida utilizando colas de mensajes POSIX

Para la versión distribuida, se comunica con el cliente a través de paso de mensajes.

- *mensaje.h*: Primero, hemos diseñado un protocolo común, que hemos añadido las definiciones de los códigos de operación (*OP_SET_VALUE*, *OP_GET_VALUE*, ...) y dos estructuras, una para la petición y otra para la respuesta. Además hemos añadido un campo para el nombre de la cola para que el servidor sepa dónde debe responder.
- *proxy-mq.c*: Este actúa como un intermediario, se encarga de empaquetar los parámetros de la solicitud en una *struct petition* y lo envía a una cola pública del servidor (*/SERVIDOR_CLAVES*), y se bloquea en espera de una respuesta en una cola privada. Además, para asegurarnos de los tamaños de los parámetros, aquí realizamos también las validaciones.
- *servidor-mq.c*: El servidor se encarga de escuchar de forma continua la cola pública, para eso, hemos implementado un bucle infinito para atender todas las peticiones. Como trabajamos con un modelo multihilo, tenemos que asegurarnos de que haya concurrencia, el hilo principal delega el procesamiento creando un hilo independiente (*pthread_create*). Para asegurar que todas las operaciones sobre ficheros del almacén se hacen de forma atómica, los hilos utilizan un cerrojo de exclusión mutua (*pthread_mutex_t*). Un hilo bloquea el *mutex* antes de invocar las funciones y lo libera justo al terminar.

2. Compilación y Ejecución

2.1. Makefile

Para optimizar el proceso de construcción de todo el proyecto, hemos creado un *Makefile* que genera tanto las bibliotecas dinámicas como los ejecutables de ambas versiones.

En cuanto a las bibliotecas dinámicas, lo hacemos de la misma manera para las dos versiones. Compilamos los ficheros *claves.c* y *proxy-mq.c* utilizando las etiquetas *-fPIC* y *-shared* para generar *libclaves.so* y *libproxyclaves.so* respectivamente.

Para generar el ejecutable (*app-cliente*) de la versión no distribuida lo hacemos con *gcc* igual que siempre, añadiendo las etiquetas *-L. -lclaves* para indicarle que busque la librería *libclaves.so* y lo una al programa. Para la versión distribuida, tenemos dos ejecutables, el servidor (*servidor-mq*) y el cliente (*app-cliente-mq*). Añadimos las mismas etiquetas, añadiendo dos más, *-lrt* para habilitar el uso de las colas de mensajes POSIX, y en el caso del ejecutable del servidor, al usar *mutex*, añadimos *-lpthread*.

2.2. Ejecución en la versión no distribuida

Al tratarse de una arquitectura monolítica, basta con ejecutar un único programa. Sin embargo, como nuestro programa usa una biblioteca dinámica propia que hemos creado en el mismo directorio, Linux no sabrá localizarla por defecto al lanzar el binario. Para solucionarlo, debemos añadir nuestro directorio actual a la variable de entorno del sistema que gestiona estas rutas. Por tanto, los pasos para la ejecución serían:

1. Exportar la ruta: **export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:.**
2. Lanzar el ejecutable: **./app-cliente**

2.3. Ejecución en la versión distribuida con colas de mensajes POSIX

Para ejecutar esta versión, tenemos que tener en cuenta lo mismo que la anterior, por lo que también debemos exportar la ruta antes, ya que necesitan localizar las bibliotecas dinámicas. Entonces, los pasos para la ejecución esta versión serían:

1. Exportar la ruta: **export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:.**
2. Arrancar el servidor: **./servidor-mq &**
Como se trata del servidor, vamos a ejecutar en background.
3. Arrancar el cliente: **./app-cliente-mq**
4. Una vez terminadas las pruebas, debemos detener el proceso del servidor que se está ejecutando en background.

3. Batería de Pruebas

Para garantizar la robustez y el correcto funcionamiento del servicio, tanto en su versión monolítica como en la distribuida, hemos diseñado una batería de pruebas automatizada e implementada íntegramente en el archivo *app-cliente.c*.

Al ejecutar el cliente, este realiza una secuencia de 15 comprobaciones divididas en dos bloques.

3.1. Pruebas de Funcionalidad (Pruebas 1-6)

Este bloque comprueba que el flujo normal de ejecución funciona correctamente.

- Prueba 1: Invocamos a la función *destroy()* para asegurar que el almacén parte de un estado vacío.
- Prueba 2: Probamos a insertar una tupla con valores válidos con la función *set_value*.
- Prueba 3: Verificamos que se puede leer correctamente la tupla insertada previamente con la función *get_value*.
- Prueba 4: Comprobamos que la tupla insertada existe con la función *exist()*.
- Prueba 5: Intentamos modificar una tupla ya previamente insertada con otro grupo de valores válidos con la función *modify_value*. Además, le asignamos a *value2* vector con el número máximo de elementos, es decir, 32.
- Prueba 6: Intentamos borrar la tupla insertada con *delete_key*.

3.2. Pruebas de Control de Errores y Límites (Pruebas 7-15)

En este segundo bloque, forzamos al sistema a fallar introduciendo datos incorrectos, verificando que bloquea la operación y nos devuelve un código de error.

- Prueba 7: Probamos a insertar una nueva tupla con una clave que ya ha sido previamente insertada.
- Pruebas 8, 10, 11: Probamos a hacer la lectura, modificación y eliminación de una clave que no se encuentra en el almacén.
- Prueba 9: Verificamos que nos devuelve error la función *exist()* al buscar una clave que no existe.
- Pruebas 12 y 13: Utilizando la función *memset* generamos una cadena de 299 caracteres y comprobamos que el programa detecta la infracción y bloquea la inserción tanto como para la clave (*key*) como para *value1*.
- Pruebas 14 y 15: Verificamos que el sistema rechaza la operación si la dimensión de *value2* (N) se encuentra en el intervalo de [1, 32].